
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We introduce FLOWTIME, a generative model that brings autoregressive flow matching to probabilistic forecasting of multivariate timeseries data. Given historical measurements and optional future covariates, FLOWTIME treats forecasting as direct sampling from a learned conditional distribution over future trajectories. The model decisively factorizes the joint distribution of future observations into a sequence of conditional densities, and it represents each density with a shared flow that transports a simple base distribution to the next-step observation distribution under the observed covariates. This construction leverages the flow matching (FM) framework to learn these transports scalably and without simulation during training. The resulting model combines the strengths of autoregressive forecasting and transport-based generative modeling in a single, compact architecture. FLOWTIME preserves the extrapolation behavior, parameter efficiency, and calibrated uncertainty estimates associated with autoregressive models, while gaining the capacity of modern flow-based generative models to represent complex multimodal conditional distributions. Experiments on stochastic dynamical systems and real-world forecasting benchmarks show that FLOWTIME is an effective and competitive approach to probabilistic timeseries forecasting.

1 INTRODUCTION

A central challenge in modern machine learning is probabilistic timeseries forecasting: estimating how system dynamics will evolve from the observations available so far. This capability underpins decision-making across scientific, industrial, and societal domains (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

Deep generative models provide a natural and increasingly powerful route to this problem (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021). They frame forecasting as conditional sampling from a future probability density, rather than as a single deterministic extrapolation. Diffusion and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have delivered strong benchmark performance in this setting (Rasul et al., 2021; Tashiro et al., 2021; Kollovieh et al., 2023; Meijer & Chen, 2024). Their gains, however, often come with substantial computational cost at training and inference time.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) offers a decisive alternative generative modeling framework that generalizes diffusion-style approaches while exposing more flexible design choices. Instead of relying on iterative stochastic denoising over a discretized trajectory, FM learns deterministic probability paths that transform arbitrary base distributions into target distributions through continuous normalizing flows. With appropriate probability paths, FM removes the need for handcrafted noise schedules and long ancestral sampling chains, yielding a more direct route to efficient training and sampling.

Recent FM-based forecasting methods (Tamir et al., 2024; Hu et al., 2024; Kollovieh et al., 2024) confirm the promise of this framework, reporting strong empirical results and better computational efficiency than diffusion-based alternatives. Yet their dominant formulation learns the conditional distribution of an entire fixed-horizon future window from a fixed-horizon context window in one shot. That non-autoregressive design enables fast generation, but it also creates a harder high-dimensional transport problem, weakens extrapolation beyond the training horizon, and can produce less calibrated uncertainty estimates. The optimization burden is explicit in (Kollovieh et al., 2024),

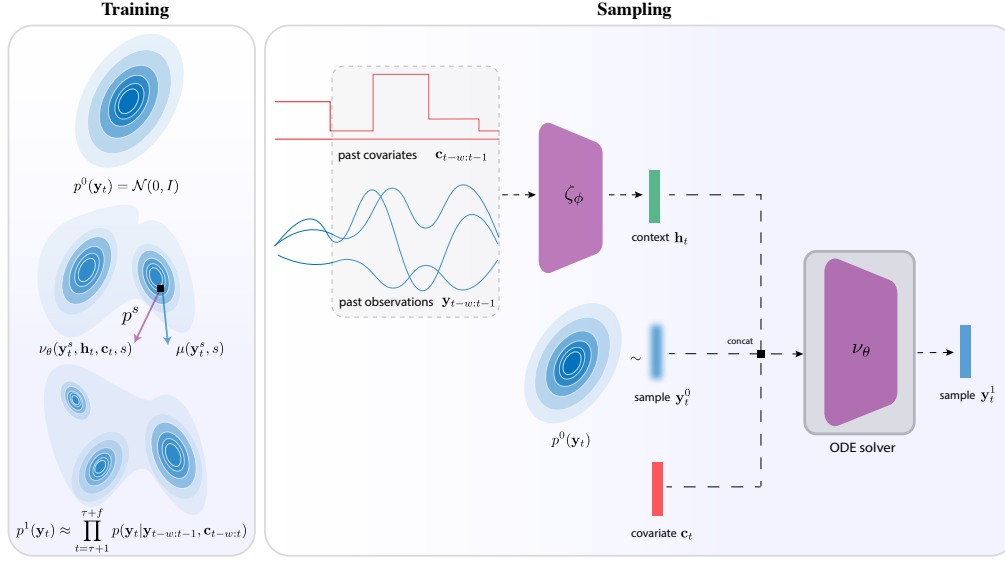


Figure 1: Overview of the training and sampling pipeline for FLOWTIME. During training, the model constructs a probability path $(p^s)_{0 \leq s \leq 1}$ from the base distribution p^0 to the target distribution p^1 , generated by a velocity field μ . A neural network ν_θ learns this velocity field from a path sample y_t^s at flow step s , a context vector h_t encoding past observations and covariates, and the current covariate c_t . At inference time, FLOWTIME samples from p^0 and integrates the learned velocity field with an ODE solver to $s = 1$, producing samples from the desired conditional target distribution.

which introduces Gaussian-process-informed priors to simplify and accelerate learning, but thereby limits the method to univariate forecasting.

We propose FLOWTIME, a direct and scalable alternative that uses FM to build probabilistic forecasters for high-dimensional multivariate timeseries without relying on informed priors. Rather than transporting an entire future window at once, FLOWTIME decomposes forecasting into a sequence of conditional next-step distributions, each conditioned on recent observations and covariates. A shared flow transports a simple base distribution to each next-observation distribution. This autoregressive flow-matching design is the central contribution: it naturally supports variable forecast horizons, strengthens extrapolation beyond the training distribution, improves uncertainty calibration, and reduces the difficulty of the learned transport problem.

We show that FLOWTIME delivers strong results on both classical stochastic dynamical systems and real-world forecasting datasets. These results establish autoregressive flow matching as a practical and effective framework for probabilistic timeseries forecasting.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a generative modeling paradigm that makes continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) practical at large scale. CNFs represent data transformations as solutions to ordinary differential equations (ODEs), yielding invertible mappings with tractable likelihoods.

As in CNFs, FM learns a time-dependent diffeomorphic map on the data space Ω , called a flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. The flow transports a sample $\mathcal{X}^0 \sim p$ from a source distribution p into a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ with $\mathcal{X}^1 \sim q$ for a target distribution q . It is defined by the following initial value problem:

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ is the vector field that sets the velocity of the flow and induces a probability path $(p^s)_{0 \leq s \leq 1}$ over Ω with $p^0 = p$. The learning problem is therefore to identify a valid vector field μ whose terminal distribution satisfies $p^1 = q$.

Unlike likelihood-trained CNFs, which require solving and differentiating through Eq. (1) during training, FM learns a parameterized vector field ν_θ by regression against vector fields associated with fixed conditional probability paths. This choice gives a tractable conditional flow matching (CFM) objective:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0,1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

After minimizing this objective, samples from q are obtained by solving Eq. (1) with the trained ν_θ . Because training avoids backpropagation through the ODE solver, FM provides stable and scalable learning for high-dimensional generative tasks, including image and video modeling (Esser et al., 2024; Polyak et al., 2024). Existing probability-path designs and conditioning strategies further show that FM unifies a broad class of transport-based generative models (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation at time τ . Given the l most recent measurements, including the current observation, $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, we seek calibrated probabilistic forecasts for the next f values $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$. The forecasts may additionally condition on observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ with $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, FLOWTIME learns to sample from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ using a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$ with m instances.

The key modeling decision is to factorize the conditional distribution autoregressively across future time steps under a Markov assumption of order w :

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ is the history size. This factorization turns the original trajectory-level forecasting problem into the task of learning the conditional distribution in Eq. (3). Rather than relying on maximum likelihood estimation, as in many autoregressive models, we use the flow matching framework (Lipman et al., 2022) to learn each conditional transport directly.

This autoregressive factorization is what gives FLOWTIME its practical advantage. It replaces one difficult high-dimensional joint distribution with a sequence of simpler conditional distributions $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$. It also permits teacher-forced training of each conditional distribution in parallel, while the Markov window w provides an explicit control knob between expressivity and computational cost. Finally, the formulation exposes uncertainty propagation through the chain of conditional distributions, yielding a clearer mechanism for calibrated probabilistic forecasts over long horizons.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For every future time step $t \in [\tau+1, \tau+f]$, FLOWTIME defines a target probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ with marginals $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, where $p_{\mathcal{D}}$ is the empirical distribution. The path is generated along s by a flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ defined through the ordinary differential equation (ODE):

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the vector field that constructs a flow satisfying the continuity equation.

To obtain a tractable objective, we model the marginal probability path as a mixture of conditional probability paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t|\mathbf{z})p(\mathbf{z})d\mathbf{z} \quad (5)$$

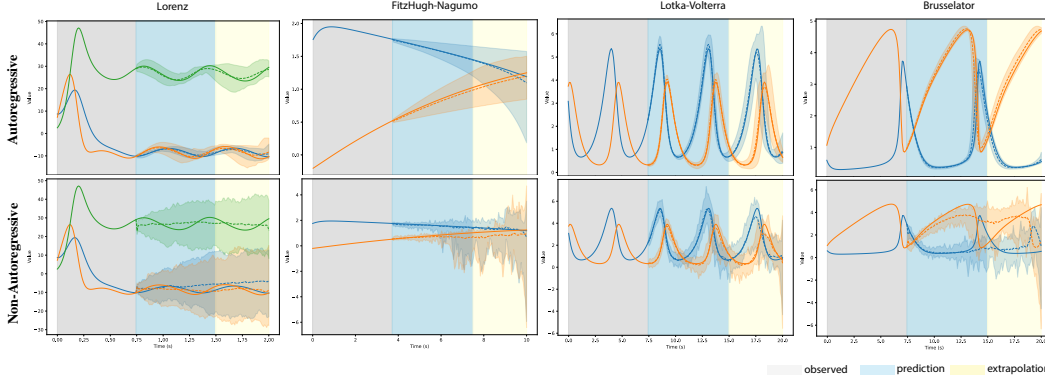


Figure 2: Forecast samples comparing autoregressive flow matching with standard flow matching on four stochastic dynamical systems. Solid lines show ground truth, while dashed lines show the predictive mean with the 95% confidence interval. The visualization highlights behavior in both the prediction and extrapolation regimes.

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning random variable sampled from an arbitrary data coupling $\pi_{0,1}$. Theorem 2 in (Lipman et al., 2022) establishes that the conditional velocity fields $\mu(\mathbf{y}_t, s|\mathbf{z})$ associated with $p^s(\mathbf{y}_t|\mathbf{z})$ recover the corresponding marginal velocity field $\mu(\mathbf{y}_t, s)$.

Following (Tong et al., 2023), we choose $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are samples from p^0 and p^1 , respectively. This gives the conditional probability path:

$$p^s(\mathbf{y}_t|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

with marginals satisfying $p^0(\mathbf{y}_t|\mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t|\mathbf{z}) = p^1(\mathbf{y}_t)$ for $\sigma^2 \rightarrow 0$. The associated conditional velocity field is the simple straight-path velocity $\mu(\mathbf{y}_t, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This choice gives straight probability paths from source to target distributions in Euclidean space and closed-form training targets, which is precisely the structure needed for efficient forecasting. Once this velocity field is learned, we sample from $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ by solving the ODE in Eq. (4) until $s = 1$.

3.2 TRAINING

As in standard FM, FLOWTIME learns the velocity field by regressing it with a neural network ν_θ with parameters θ under the conditional flow matching objective in Eq. 2. To make the objective autoregressive and context-aware, we define a context vector $\mathbf{h}_t \in \mathbb{R}^h$ that encodes the context window at time t :

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural network with parameters ϕ . The resulting learning objective is:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t|\mathbf{z})} \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

We optimize this objective by sampling an observation time t , its context window, and its covariates from $\mathcal{D}$, then jointly updating ϕ and θ by stochastic gradient descent on $\mathcal{L}(\theta, \phi)$. This training procedure is equivalent in spirit to teacher forcing (Williams & Zipser, 1989): during training, the model receives ground-truth past observations rather than its own predictions. As a result, training remains parallel across time steps and avoids compounding rollout error. Algorithm 1 summarizes autoregressive flow matching training.

3.3 INFERENCE

After training, FLOWTIME produces probabilistic predictions for new observations $\mathbf{Y}_t$ and covariates $\mathbf{C}$ over the next f timestamps. Algorithm 2 performs autoregressive sampling with a rolling window: each generated sample becomes part of the context for subsequent steps. Repeating this procedure yields empirical predictive quantiles and a direct estimate of forecast uncertainty.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
- 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
- 5: Compute target: $\mu(\mathbf{y}_t^s, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
- 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
- 7: Update θ, ϕ
- 8: **end while**
- 9: **return** ν_θ, ζ_ϕ

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s), \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0, \quad s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

4 EXPERIMENTS

We evaluate FLOWTIME on both controlled stochastic dynamical systems and established real-world forecasting datasets. The experiments test the main claims of the method: autoregressive flow matching improves extrapolation and calibration relative to a non-autoregressive FM ablation, and it remains competitive with strong forecasting baselines on practical benchmarks.

4.1 DYNAMICAL SYSTEMS

Data We first evaluate FLOWTIME on five stochastic dynamical systems that provide a stringent test of extrapolation: stochastic versions of the Lorenz system (Lorenz, 1963), FitzHugh–Nagumo model (FitzHugh, 1961), Lotka-Volterra system Lotka (1925), Brusselator (Lefever & Prigogine, 1968), and Van der Pol oscillator (van der Pol, 1926). For each system, we generate trajectories by numerically solving the corresponding stochastic differential equations from randomly sampled initial conditions and Brownian motion realizations. Appendix A.1 gives the full data-generation protocol.

Implementation details For these experiments, we instantiate ζ_ϕ as a two-layer bi-directional LSTM with 64 hidden units and ν_θ as an MLP with three hidden layers of size 64. Fourier positional embeddings encode the flow step s into a 16-dimensional vector using a fixed set of frequencies, and the context length w equals the prediction length. We train with stochastic gradient descent using Adam with learning rate 0.003 and batch size 128. Training to convergence took ~ 7 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines To isolate the contribution of the autoregressive factorization, we compare FLOWTIME with a non-autoregressive flow-matching baseline that directly models the joint future distribution conditioned on past observations and covariates, i.e. $p^1 = p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. This baseline is the critical ablation: it keeps the FM machinery while removing the factorization introduced by FLOWTIME. The formulation closely resembles the method proposed in (Kollovich et al., 2024); Appendix B provides details.

Evaluation To directly measure extrapolation beyond the training distribution, we partition each trajectory into three segments: (1) an observation window used as historical input; (2) a prediction

window available during training as the target; and (3) an extrapolation window that remains unseen during training. We evaluate uncertainty quality with the continuous rank probability score (CRPS) (Winkler et al., 1996) and point-forecast accuracy with normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006).

Table 1: Autoregressive FLOWTIME versus the non-autoregressive (Non-AR) flow-matching ablation on five stochastic dynamical systems. CRPS and NRMSE are reported on the test set for both prediction and extrapolation regimes. Values are means and standard deviations across five random seeds.

System	Factorization	Prediction		Extrapolation	
		CRPS ↓	NRMSE ↓	CRPS ↓	NRMSE ↓
Lorenz	Non-AR	0.147±0.012	0.242±0.018	0.120±0.009	0.301±0.024
	FLOWTIME	0.090±0.008	0.017±0.003	0.137±0.011	0.048±0.006
FitzHugh-Nagumo	Non-AR	0.165±0.014	0.131±0.011	0.214±0.016	0.306±0.022
	FLOWTIME	0.144±0.013	0.090±0.007	0.146±0.012	0.278±0.019
Lotka-Volterra	Non-AR	0.130±0.011	0.266±0.019	0.136±0.012	0.502±0.033
	FLOWTIME	0.126±0.010	0.189±0.015	0.123±0.009	0.339±0.026
Brusselator	Non-AR	0.217±0.016	0.658±0.042	0.238±0.019	1.046±0.057
	FLOWTIME	0.125±0.011	0.031±0.004	0.109±0.008	0.023±0.003
Van der Pol	Non-AR	0.143±0.012	0.161±0.014	0.211±0.018	0.510±0.031
	FLOWTIME	0.225±0.017	0.051±0.005	0.282±0.021	0.101±0.009

Results Table 1 provides strong evidence that the autoregressive factorization in FLOWTIME improves forecasting on stochastic dynamical systems. In the prediction regime, FLOWTIME achieves lower NRMSE than the non-AR baseline on all five systems, with especially large reductions on Lorenz (0.017 vs 0.242) and Brusselator (0.031 vs 0.658), corresponding to error reductions of 93% and 95%. It also achieves better CRPS on four of the five systems; only Van der Pol favors the non-AR baseline on CRPS. The advantage becomes even more consequential in extrapolation, where FLOWTIME keeps the best NRMSE on every system, demonstrating stronger generalization beyond the training distribution. The Brusselator result is the clearest case: autoregressive factorization reduces extrapolation NRMSE by 98% (0.023 vs 1.046). These results support the central claim that explicitly modeling temporal dependence through autoregressive flow matching captures stochastic dynamics more effectively than transporting the full future window at once.

4.2 REAL-WORLD DATASETS

Data We next evaluate FLOWTIME on GluonTS (Alexandrov et al., 2020) benchmarks covering univariate and multivariate timeseries at hourly and daily frequencies. The datasets are Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019). Appendix A.2 provides the dataset details.

Implementation details We instantiate ζ_ϕ as a three-layer bi-directional LSTM with 64 hidden units. The velocity network ν_θ is a residual neural network with five residual blocks implemented by 1-D convolutional layers with gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020). This architecture follows the design used in (Rasul et al., 2021) but uses fewer residual blocks. Fourier positional embeddings encode the flow step s into a 32-dimensional vector with fixed frequencies, and the same embedding strategy encodes time-dependent covariates $\mathbf{c}$. We train with Adam using learning rate 0.001 and batch size 128; depending on the dataset, convergence took 40 – 100 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines We compare FLOWTIME against established statistical, deep learning, diffusion, and flow-matching baselines. The statistical baselines are Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008); the deep learning baselines include DeepAR (Salinas et al., 2020) and

WaveNet (Van Den Oord et al., 2016). We also include diffusion-based methods TSDiff (Kollovich et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021), together with the flow-matching baseline TSFlow (Kollovich et al., 2024).

Table 2: Test-set CRPS comparison on five real-world forecasting datasets, where lower is better. FLOWTIME results report the mean and standard error over five independent retraining and evaluation runs. Baseline results are from (Kollovich et al., 2024). Best scores are shown in **bold**; second-best scores are underlined.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060±0.000	0.013±0.000	0.512±0.000	0.221±0.000	0.423±0.000
ARIMA	0.344±0.000	0.013±0.000	0.558±0.003	0.486±0.000	0.654±0.000
ETS	0.056±0.000	0.008±0.000	0.550±0.000	0.492±0.000	0.651±0.000
DeepAR	0.051±0.000	0.013±0.004	0.429±0.015	0.103±0.000	0.215±0.003
WaveNet	0.058±0.008	0.012±0.001	0.360±0.000	0.099±0.000	0.207 ±0.000
CSDI	0.051±0.000	0.013±0.000	0.360±0.000	0.152±0.000	0.318±0.012
SSSD	0.048±0.000	<u>0.010</u> ±0.000	0.354±0.024	0.107±0.002	<u>0.209</u> ±0.000
TSFlow	<u>0.045</u> ±0.001	0.009 ±0.001	<u>0.343</u> ±0.002	0.083 ±0.000	0.227±0.000
FLOWTIME	0.042 ±0.001	0.009 ±0.001	0.284 ±0.002	<u>0.089</u> ±0.000	0.243±0.002

Results Table 2 shows that FLOWTIME is competitive with the strongest available real-world forecasting baselines. It achieves state-of-the-art CRPS on three of the five datasets: Electricity, Exchange, and Solar. On Electricity, FLOWTIME reaches 0.042 CRPS, improving over the second-best TSFlow result of 0.045 by 6.7%. On Exchange, it matches TSFlow at 0.009 CRPS and clearly surpasses the remaining statistical and deep learning baselines. On Solar, the improvement is substantial: FLOWTIME obtains 0.284 CRPS, a 17.2% error reduction relative to TSFlow at 0.343. The method ranks second on Traffic, with 0.089 CRPS versus TSFlow’s 0.083, and remains ahead of the other statistical and deep learning baselines. On Wikipedia, FLOWTIME is competitive at 0.243 CRPS, trailing WaveNet (0.207) and SSSD (0.209). Overall, these results show that autoregressive flow matching is not only effective in controlled dynamical systems, but also delivers state-of-the-art or competitive performance across diverse real-world forecasting tasks.

5 DISCUSSION

We introduced FLOWTIME, a probabilistic forecasting model that unifies autoregressive modeling with flow matching to generate accurate and well-calibrated forecasts for multivariate timeseries. Its central design choice is to factorize trajectory forecasting into next-step conditional transports, thereby preserving temporal structure while retaining the expressivity of flow-based generative modeling. The empirical evidence supports this design. Across stochastic dynamical systems, FLOWTIME consistently improves NRMSE over the non-autoregressive FM ablation, including in extrapolation windows never seen during training. Across real-world datasets, it achieves state-of-the-art CRPS on three benchmarks and remains competitive on the others. These results show that the autoregressive decomposition is not merely an architectural variant: it materially improves uncertainty propagation, extrapolation behavior, and forecasting accuracy when compared with modeling entire future windows simultaneously.

The method also has clear limitations. Autoregressive sampling is sequential at inference time, so it introduces overhead relative to fully parallel non-autoregressive models and may be less suitable for strict real-time settings. The current implementation also applies flow matching directly in data space, which may be suboptimal for high-dimensional or partially observed systems. These limitations identify concrete extensions: latent-space flow matching, irregularly sampled observations, and broader deployment in scientific modeling and uncertainty-aware decision-making. Taken together, the results position FLOWTIME as a strong step forward for probabilistic timeseries forecasting with flow-based generative models.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.
- Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.
- Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint arXiv:1605.06432*, 2016.
- Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for probabilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–28364, 2023.

432 Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow
433 matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth*
434 *International Conference on Learning Representations*, 2024.

435 Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile
436 diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.

437 Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term
438 temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference*
439 *on Research & Development in Information Retrieval*, pp. 95–104, 2018.

440 R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal*
441 *of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.

442 Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical*
443 *Transactions of the Royal Society A*, 379(2194):20200209, 2021.

444 Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow match-
445 ing for generative modeling. In *The Eleventh International Conference on Learning Representa-*
446 *tions*, 2022.

447 Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data
448 with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.

449 Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.

450 Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.

451 Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv*
452 *preprint arXiv:2401.03006*, 2024.

453 Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv
454 Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media founda-
455 tion models. *arXiv e-prints*, pp. arXiv–2410, 2024.

456 Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Mul-
457 tivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint*
458 *arXiv:2002.06103*, 2020.

459 Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising dif-
460 fusion models for multivariate probabilistic time series forecasting. In *International Conference*
461 *on Machine Learning*, pp. 8857–8868. PMLR, 2021.

462 David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus.
463 High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances*
464 *in Neural Information Processing Systems*, 32, 2019.

465 David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic
466 forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):
467 1181–1191, 2020.

468 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
469 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learn-*
470 *ing*, pp. 2256–2265. PMLR, 2015.

471 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
472 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint*
473 *arXiv:2011.13456*, 2020.

474 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
475 ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-*
476 *ative Modeling*, 2024.

486 Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
487 diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-*
488 *cessing Systems*, 34:24804–24816, 2021.

489 Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguët, Yanlei Zhang, Jarrod Rector-
490 Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
491 with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

492 Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves,
493 Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for
494 raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.

495 B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical*
496 *Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.

497 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
498 neural networks. *Neural Computation*, 1(2):270–280, 1989.

499 Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B
500 Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring
501 rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.

502 Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time
503 series foundation models: Generalizing time series representation with large language model. In
504 *ArXiv:2104.00164*, 2024.

505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539

A DATASETS

A.1 DYNAMICAL SYSTEMS

We generated training and testing data for five classical stochastic dynamical systems: Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator, and Van der Pol. For each system, we numerically integrated the corresponding stochastic differential equation (SDE), which takes the general form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

where f is the deterministic drift, Θ are the system parameters, σ is a constant diffusion term, and $\mathbf{W}(t)$ is standard Brownian motion. For each system, we sampled 2400 random initial conditions from the uniform range specified in Table 3 and generated one trajectory per initial condition using the Euler-Heun method with fixed step size. Each trajectory contains 200 equally spaced time points. We use the first 75 points as observed data, the next 75 as the prediction region, and the final 50 as the extrapolation region. Of the 2400 generated trajectories, 2000 are used for training and 400 for testing.

Table 3: Stochastic dynamical systems used to evaluate extrapolation and uncertainty quality. $\mathbf{f}(\mathbf{x})$ denotes the drift function, Θ the system parameters, σ the constant diffusion term, and $p(\mathbf{x}(0))$ the initial condition distribution. Each system is integrated over $[t_0, t_1]$ with 200 Euler-Heun steps.

System	$f(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh-Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka-Volterra	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\beta = 0.9$			
		$\gamma = 0.8$			
Brusselator	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$A = 1.0$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_2 = B x_1 - x_1^2 x_2$	$B = 3.0$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

We use five GluonTS datasets (Alexandrov et al., 2020) to evaluate real-world forecasting performance. All datasets use the preprocessing protocol of (Salinas et al., 2019), and Table 4 summarizes the resulting dataset properties.

Table 4: Real-world dataset properties used in the forecasting experiments: dimension n , domain Ω , sampling frequency, total training timesteps, and prediction length.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting We use the same forecasting setup as Section 3: given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, the goal is to model $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. The non-autoregressive baseline deliberately removes the central factorization of FLOWTIME and instead models the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ as one joint random variable:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves temporal correlations across all future time steps, but it forces the velocity model to learn a substantially higher-dimensional distribution.

Training The baseline constructs a probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from the prior $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to the target $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is block-diagonal. This prior can be interpreted as independent Brownian motion processes per dimension, with Brownian motion defined as a stochastic process $W: [\tau + 1, \tau + f] \rightarrow \mathbb{R}^n$.

The conditional probability path is a linear interpolation bridge with Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field is:

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

When $\sigma^2 \rightarrow 0$, this reduces to the straight-path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. The neural velocity field ν_θ acts on the full future trajectory. Its context encoder follows the same overall role as Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now processes both past and future covariates. The baseline training objective is:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

where ν_θ is implemented as a sequential neural network, such as a Transformer or RNN, that processes the complete trajectory. Sampling then requires solving the trajectory-level ODE:

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

with numerical solvers. The procedure produces joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ without using the autoregressive decomposition.

Implementation Details We use this baseline for the dynamical-system forecasting experiments. In this setup, ν_θ is a 4-layer bi-directional LSTM with 128 hidden units. All other training parameters match Section 4.1. Training to convergence took ~ 2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant The non-autoregressive baseline differs from FLOWTIME in the core modeling stance. FLOWTIME factorizes the joint distribution with a Markovian decomposition across time steps, whereas the baseline models the full future trajectory $\mathbf{Y}_f$ as a single high-dimensional random variable. This preserves cross-temporal dependencies, but it also requires learning a more complex distribution over $\mathbb{R}^{n \times f}$. The computational profile also changes sharply. The non-autoregressive version processes entire trajectories through sequential neural networks for the velocity field, which enables parallel generation of future time points but increases memory requirements. The uncertainty mechanisms differ as well: FLOWTIME explicitly propagates predictive uncertainty through a chain of conditional distributions, while the non-autoregressive method captures joint uncertainty through trajectory-level sampling without an explicit model of error accumulation. These trade-offs clarify the empirical comparison: the autoregressive variant offers interpretable uncertainty propagation and efficient window-based computation under a Markov assumption, whereas the non-autoregressive baseline retains full trajectory-level modeling at higher computational and optimization cost.

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ
1: **while** not converged **do**
2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
4: Sample $s \sim \mathcal{U}(0, 1)$
5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
6: Compute target $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$
7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ
1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
4: Return $\psi(\mathbf{Y}_f, 1)$

C EVALUATION METRICS

We evaluate forecast quality with two complementary metrics applied to an n -dimensional future time series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ and its predictive distribution or model samples: the Continuous Ranked Probability Score (CRPS) for probabilistic calibration and the Normalized Root Mean Squared Error (NRMSE) for point accuracy.

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

CRPS is a proper scoring rule that directly measures the quality of a probabilistic forecast. For a univariate random variable X with cumulative distribution function (CDF) F_X and observed value x , CRPS is defined as:

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. CRPS evaluates alignment between the full predictive distribution and the observation x , so lower values indicate sharper and better-calibrated probabilistic forecasts.

For time series forecasting, we compute CRPS at each forecast horizon and average over the horizon:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ is the predicted CDF, or the empirical CDF from samples, at forecast horizon k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

Root Mean Squared Error (RMSE) measures point-forecast accuracy. For predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, RMSE is:

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

We report Normalized RMSE (NRMSE) to make this error scale-invariant and comparable across datasets with different magnitudes. We normalize by the standard deviation of the ground-truth

series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations. Other applications may normalize by the data range ($\max(y) - \min(y)$), but the interpretation remains the same: lower NRMSE indicates more accurate predictions relative to the variability of the series.